Day 76 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 7 reference

Watch: A Dive Into Multihead Attention, Self-Attention and Cross-Attention (Machine Learning Studio) | Visual Guide to Transformer Neural Networks, Episode 2 (Hedu AI)

`day76_multi_head_attention.py`, segment by segment, in the order `main()` uses things. Day 75's attention function and Day 72's alignment_metrics are copied verbatim from the exercise scripts and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: m = examples · n = tokens (16 pairs + 1 query = 17) · d = 64, the model size · h = number of heads · d/h = one head's size · A = attention weights

Segment 0 — Setup

Settings

CODE

```
N_KEYS = 32
N_PAIRS = 16
D = 64
HEADS = (1, 2, 4, 8)
SEEDS = [0, 1, 2]
STEPS = 2000
CHECKS = (100, 250, 500, 1000, 2000)
COLORS = {1: "#d62728", 2: "#1f77b4", 4: "#2ca02c", 8: "#9467bd"}
```

- `D = 64` — every model has the same size; only how it is cut into heads changes.
- `HEADS = (1, 2, 4, 8)` — head sizes 64, 32, 16 and 8: each divides 64 exactly.
- `COLORS = {1: ..., 2: ...}` — a dictionary from head count to colour, used in every plot.

Segment 1 — Multi-head attention

split_heads and merge_heads

MATH

$(m, n, d) \rightarrow (m, n, h, d/h) \rightarrow (m, h, n, d/h) \rightarrow (m{\cdot}h, n, d/h)$

CODE

```
def split_heads(T, h):
    m, n, d = T.shape
    return T.reshape(m, n, h, d // h).transpose(0, 2, 1, 3).reshape(m * h, n, d // h)
def merge_heads(T, h):
    mh, n, dk = T.shape
    return T.reshape(mh // h, h, n, dk).transpose(0, 2, 1, 3).reshape(mh // h, n, h * dk)
```

- `T.reshape(m, n, h, d // h)` — the last axis of d numbers becomes h groups of d/h: group i is columns i*d/h onward.
- `.transpose(0, 2, 1, 3)` — reorders the axes to (m, h, n, d/h): the head axis moves in front of the positions.
- `.reshape(m * h, n, d // h)` — merges examples and heads into one batch axis, so Day 75's 3-axis function runs every head.

- `mh // h` — integer division: the number of examples, from examples times heads.

multi_head_attention

MATH

$$output = [head_1 ; ... ; head_h] W_o, \quad head_i = softmax(Q_i K_i^T / \sqrt{(d/h)}) V_i$$

CODE

```
def multi_head_attention(X, P, h, mask=None):
    Q, K, V = (split_heads(X @ P[w], h) for w in ("Wq", "Wk", "Wv"))
    Zh, A = scaled_dot_product_attention(Q, K, V, mask)
    Z = merge_heads(Zh, h)
    return Z @ P["Wo"], dict(X=X, Q=Q, K=K, V=V, A=A, Z=Z)
```

- `(split_heads(X @ P[w], h) for w in (...))` — a generator expression, unpacked into three names: one projection per weight name.
- `scaled_dot_product_attention(Q, K, V, mask)` — Day 75's exercise function, unchanged.
- `Z @ P["Wo"]` — the output projection mixes the heads' outputs.

multi_head_attention_backward

MATH

$$dW_o = Z^T dO; \quad dX = dQ\, W_q^T + dK\, W_k^T + dV\, W_v^T$$

CODE

```
def multi_head_attention_backward(dO, P, h, c, scores=True):
    flat = lambda T: T.reshape(-1, T.shape[-1])
    g = dict(Wo=flat(c["Z"]).T @ flat(dO))
    dZh = split_heads(dO @ P["Wo"].T, h)
    dQ, dK, dV = scaled_dot_product_attention_backward(dZh, c["Q"], c["K"], c["V"], c["A"])
    if not scores:
        dQ, dK = np.zeros_like(dQ), np.zeros_like(dK)
    dQ, dK, dV = merge_heads(dQ, h), merge_heads(dK, h), merge_heads(dV, h)
    X = c["X"]
    g.update(Wq=flat(X).T @ flat(dQ), Wk=flat(X).T @ flat(dK), Wv=flat(X).T @ flat(dV))
    dX = dQ @ P["Wq"].T + dK @ P["Wk"].T + dV @ P["Wv"].T
    return dX, g
```

- `flat = lambda T: T.reshape(-1, T.shape[-1])` — a one-line function: every position becomes one row.
- `split_heads(dO @ P["Wo"].T, h)` — back through W_o, then cut the gradient into heads the same way.
- `if not scores: dQ, dK = zeros` — Section 5: forced weights do not depend on the queries or keys.
- `g.update(Wq=..., Wk=..., Wv=...)` — adds three entries to the gradient dictionary at once.

Segment 2 — The task and the model

make_batch: two asked keys

CODE

```
def make_batch(rng, m, two=True, n_keys=N_KEYS, n_pairs=N_PAIRS):
    keys = np.array([rng.permutation(n_keys)[:n_pairs] for _ in range(m)])
    vals = rng.randint(0, n_keys, size=(m, n_pairs))
    pa = rng.randint(0, n_pairs, size=m)
    pb = (pa + rng.randint(1, n_pairs, size=m)) % n_pairs
    if not two:
        pb = pa
    r = np.arange(m)
    return keys, vals, keys[r, pa], keys[r, pb], vals[r, pa], vals[r, pb], pa, pb
```

- `(pa + rng.randint(1, n_pairs, size=m)) % n_pairs` — adds 1..15 and wraps around: the second pair is never the first.
- `if not two: pb = pa` — the control asks about the same pair twice.
- `keys[r, pa]` — fancy indexing: in example r take column pa[r].

TwoKeyRecall.forward

MATH

$$x_{query} = E_a[\text{first key}] + E_b[\text{second key}];\quad p_a = softmax(o\,U_a + b_a),\quad p_b = softmax(o\,U_b + b_b)$$

CODE

```
def forward(self, keys, vals, qa, qb, force=None):
    P = self.P
    pairs = P["Ek"][keys] + P["Ev"][vals]
    query = P["Ea"][qa] + P["Eb"][qb]
    X = np.concatenate([pairs, query[:, None, :]], axis=1)
    out, c = multi_head_attention(X, P, self.h)
    if force is not None:
        c = self.forced_read(c, force)
        out = c["Z"] @ P["Wo"]
    o = out[:, -1]
    pa, pb = softmax(o @ P["Ua"] + P["ba"]), softmax(o @ P["Ub"] + P["bb"])
    self.cache = dict(c=c, keys=keys, vals=vals, qa=qa, qb=qb, o=o, pa=pa, pb=pb, forced=force is
        not None)
    return pa, pb
```

- `np.concatenate([pairs, query[:, None, :]], axis=1)` — the query becomes token 17; [:, None, :] gives it a positions axis of size 1.
- `out[:, -1]` — index -1: the last token, the query.
- `force is not None` — the cache stores a boolean, so the backward pass knows whether the weights were forced.

forced_read

CODE

```
def forced_read(self, c, force):
    A = c["A"].copy()
    m = len(force[0]); r = np.arange(m)
    A = A.reshape(m, self.h, A.shape[1], A.shape[2])
    A[:, :, -1, :] = 0.0
    if self.h == 1:
        A[r, 0, -1, force[0]] += 0.5; A[r, 0, -1, force[1]] += 0.5
    else:
        A[r, 0, -1, force[0]] = 1.0
        A[r[:, None], np.arange(1, self.h)[None, :], -1, force[1][:, None]] = 1.0
    A = A.reshape(m * self.h, A.shape[2], A.shape[3])
    Zh = np.einsum("mij,mjd->mid", A, c["V"])
    return dict(c, A=A, Z=merge_heads(Zh, self.h))
```

- `A[r, 0, -1, force[0]] += 0.5` — half the weight on pair a; the statement after the semicolon puts the other half on pair b.
- `A[r[:, None], np.arange(1, self.h)[None, :], -1, force[1][:, None]]` — broadcast indexing: every example and every head after the first.
- `dict(c, A=A, Z=...)` — a copy of the cache dictionary with two entries replaced.

loss_and_grads

MATH

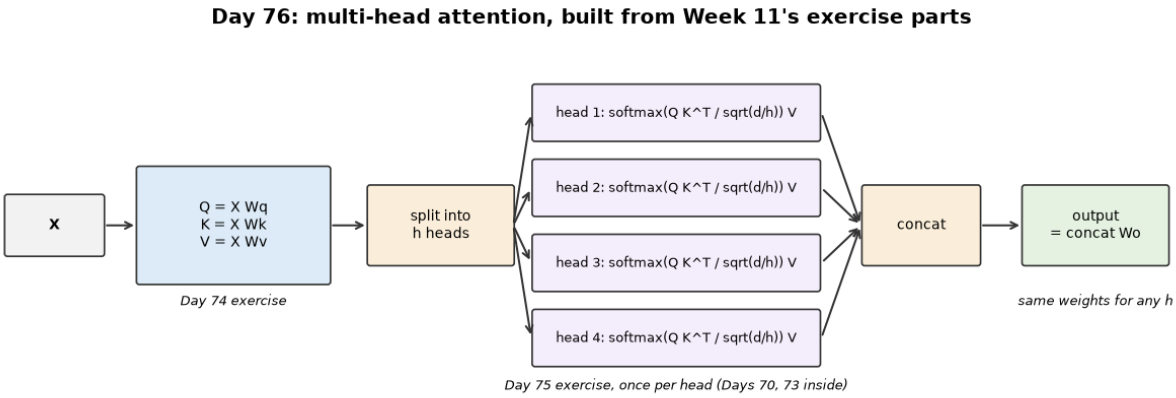
$$L = \tfrac{1}{2}\left(-\log p_a[y_a] - \log p_b[y_b] \right), \text{averaged over the batch}$$

CODE

```
def loss_and_grads(self, keys, vals, qa, qb, ya, yb, force=None):
    P = self.P
    pa, pb = self.forward(keys, vals, qa, qb, force)
    C = self.cache
    m = len(ya); r = np.arange(m)
    loss = -(np.log(pa[r, ya] + 1e-12) + np.log(pb[r, yb] + 1e-12)).mean() / 2
    da = pa.copy(); da[r, ya] -= 1; da /= 2 * m
    db = pb.copy(); db[r, yb] -= 1; db /= 2 * m
    g = dict(Ua=C["o"].T @ da, ba=da.sum(axis=0), Ub=C["o"].T @ db, bb=db.sum(axis=0))
    dout = np.zeros(C["c"]["X"].shape)
    dout[:, -1] = da @ P["Ua"].T + db @ P["Ub"].T
    dX, ga = multi_head_attention_backward(dout, P, self.h, C["c"], scores=not C["forced"])
    g.update(ga)
    for k in ("Ek", "Ev", "Ea", "Eb"):
        g[k] = np.zeros_like(P[k])
    np.add.at(g["Ek"], C["keys"], dX[:, :-1]); np.add.at(g["Ev"], C["vals"], dX[:, :-1])
    np.add.at(g["Ea"], C["qa"], dX[:, -1]); np.add.at(g["Eb"], C["qb"], dX[:, -1])
    return loss, g
```

- `dout[:, -1] = ...` — only the query token's output reaches a loss, so every other token's gradient stays 0.
- `np.add.at(g["Ea"], C["qa"], dX[:, -1])` — the query's first-key table collects from the query token only.

SYSTEM DIAGRAM



Multi-head attention, with the week's parts named.

Segment 3 — Experiments

evaluate, train and the cache

CODE

```
def evaluate(net, two=True, force=False):
    keys, vals, qa, qb, ya, yb, pa_, pb_ = TESTS[two]
    pa, pb = net.forward(keys, vals, qa, qb, (pa_, pb_) if force else None)
    a, b = pa.argmax(axis=1) == ya, pb.argmax(axis=1) == yb
    return np.array([a.mean(), b.mean(), (a & b).mean()])
def train(heads, two, seed, d=D, steps=STEPS, force=False):
    net = TwoKeyRecall(d, heads, seed=seed)
    rng = np.random.RandomState(seed + 100)
    curve = []
    checks = [c * steps // STEPS for c in CHECKS]
    for step in range(1, steps + 1):
        keys, vals, qa, qb, ya, yb, pa, pb = make_batch(rng, 64, two)
        net.train_step(keys, vals, qa, qb, ya, yb, (pa, pb) if force else None)
        if step in checks:
            curve.append(evaluate(net, two, force))
    return net, np.array(curve)
RUNS = {}
def train_cached(*args, **kwargs):
    key = (args, tuple(sorted(kwargs.items())))
    if key not in RUNS:
        RUNS[key] = train(*args, **kwargs)
    return RUNS[key]
```

- `np.array([a.mean(), b.mean(), (a & b).mean()])` — & on boolean arrays: both answers right.
- `[c * steps // STEPS for c in CHECKS]` — stretches the checkpoints for the 8,000-step run (400, 1000, ...).
- `tuple(sorted(kwargs.items()))` — keyword arguments in a fixed order, so equal runs get equal keys.

demo_parts: one call against a loop

CODE

```
for h in (1, 2, 4):
    out, _ = multi_head_attention(X, P, h)
    dk = 8 // h
    Q, K, V = X @ P["Wq"], X @ P["Wk"], X @ P["Wv"]
    parts = [scaled_dot_product_attention(Q[..., i * dk:(i + 1) * dk], K[..., i * dk:(i + 1) *
        dk],
                                          V[..., i * dk:(i + 1) * dk])[0] for i in range(h)]
    loop = np.concatenate(parts, axis=-1) @ P["Wo"]
    print(f"{h:>6}{dk:>11}{np.abs(out - loop).max():>36.2e}")
```

- `Q[..., i * dk:(i + 1) * dk]` — ... means every leading axis; only the last axis is sliced.
- `np.concatenate(parts, axis=-1)` — the heads side by side along the last axis.

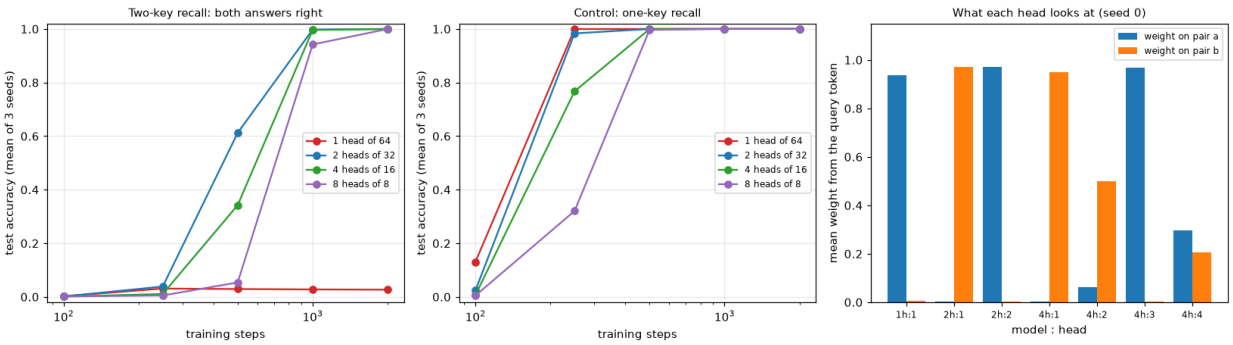
demo_heads: Day 72's measurements per head

CODE

```
A = net.cache["c"]["A"].reshape(len(qa), h, N_PAIRS + 1, N_PAIRS + 1)[:, :, -1, :]
heads_w[h] = []
for i in range(h):
    ma, mb = alignment_metrics(A[:, i], pa), alignment_metrics(A[:, i], pb)
    heads_w[h].append((ma[1], mb[1]))
```

- `[:, :, -1, :]` — the query token's row of every head: (examples, heads, 17).
- `alignment_metrics(A[:, i], pa)` — Day 72's exercise function with target = where pair a sits.

REAL GRAPH



Real graph from the script: two-key recall, the one-key control, and what each head looks at.

Quick Syntax Glossary — new or reinforced constructs

- `T.reshape(...).transpose(...)` — regroup a flat axis, then reorder axes, without copying numbers by hand.
- `a, b, c = (f(x) for x in xs)` — unpack a generator expression into names.
- `X[..., i:j]` — Ellipsis: all leading axes, slice only the last.
- `A[r[:, None], cols[None, :], -1, idx[:, None]]` — broadcast fancy indexing over two axes at once.
- `dict(d, key=value)` — a copy of d with some entries replaced.
- `a & b` — elementwise AND of boolean arrays.
- `n // h` — integer division.

End of Day 76 syntax reference. For the experiments and what they showed, see the Day 76 complete guide.